P2021R0
Negative zero strikes again

Published Proposal, 2020-01-05

This version:
http://fmt.dev/P2021R0.html

Author:
Victor Zverovich

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

"Your scientists were so preoccupied with whether or not they could,
they didn't stop to think if they should."
— Dr. Ian Malcolm

National body comment [US227] proposed adding a new feature to the C++20 formatting library, namely a new format specifier to suppress the output of a minus sign when formatting a negative zero or a small negative value. This was an attempt to revisit [P1496] that was previously reviewed by LEWG in Kona and rejected with no consensus for change. R1 of this paper presented in Belfast after the C++20 feature freeze and didn't provide any new information, only gave more examples showing (unfortunately incorrectly) that suppression applies to small negative values rounding to zero. This was also discussed in Kona as reflected in the meeting notes.

Sadly the analysis in [P1496] is severely lacking: examples are misleading, it doesn't explore performance implications of the change, interactions with other format specifications, interaction with the locale and there is no implementation or usage experience, so it's extremely surprising that the feature is being seriously considered for C++20. There was also a piece of incorrect information about performance implications if the suppression was not a built-in specifier. In this paper we provide a somewhat deeper analysis of the issue and clarify the following:

- there is almost zero evidence that this is a problem worth solving,
- sign suppression will add an overhead to calls to `std::format` and uses of built-in formatter specializations even if not used,
- the new feature will complicate the grammar and add conceptual overhead,
- negative zero doesn't arise as a result of rounding in all but fixed format and therefore can be trivially and more efficiently suppressed without a new specifier,
- it can be easily and efficiently implemented via the extension API,
- as a user-facing feature it should be locale-specific if implemented.

§ 1. Is negative zero a problem?

High school math teaches us that -0 = 0. So does minus in front of zero actually cause any problems in practice? Unfortunately P1496 only provides anecdotal evidence from one of the authors of the paper and even original authors of P1496R0 disagreed whether it's worth exploring considering problematic performance and complexity tradeoffs. P1496 casually extrapolates to millions of affected users but as we know the plural of anecdote is not data.

In order to find some real evidence we looked at the fmt library ([FMT]) and Folly which includes a similar formatting facility ([FOLLY-FORMAT]). Both of these are popular open-source formatting

libraries that have been around for 7 years each and are widely deployed in software used by billions of people. A thorough search of the issue trackers of these libraries for issues related to zero has revealed only a single feature request related to suppression of negative zero. It has been easily addressed via an extension API without introducing a new specifier to avoid an overhead when not using the feature. Another issue that mentioned a negative zero was asking for the output of -0.01 with zero precision to be `"-0"` instead of `"-0.0"` to be consistent with `printf`. Here `'-'` is a desirable part of the output. So the evidence in support of adding this feature is not zero but close to it and shows that it is easily solvable by other means.

Another observation is that negative zero is not the only case that users may find unusual. Why does `"-0.0"` deserve more attention than the following incomplete list of "interesting" cases:

- `"1.000000e+42"`
- `"+00"` in `"1.000000e+00"` which is the default "scientific" format of 1.0
- `"nan"`
- `"inf"`

If the desire is to make floating-point numbers be presented in a user-friendly form, just suppressing a sign is obviously not enough. Therefore even if we assume that it's an important issue despite very little evidence it only addresses one of multiple cases. There are more comprehensive solutions to this problem such as [D3-FORMAT] and they can be implemented using the extension API without introducing new format specifiers.

We also looked at standard formatting facilities of popular programming languages (C, C++, Java, Python) and haven't found sign suppression available in any of them.

§ 2. Performance

P1496 doesn't discuss performance implications of the change it proposes which is very unfortunate because adding a built-in formatting option has a nontrivial cost. The change is deceptively simple: just add a new character to the syntax and somehow make it suppress the sign. What it means in practice is the following:

- Make the parser for built-in format specifiers recognize the new option
- Add a new piece of state to formatter specialization for floating-point types or all built-in types (depending on the implementation)
- Add a new runtime check every time a floating-point number is formatted and perform additional work to suppress the minus sign

Note that all of these except the actual suppression add an overhead even if you don't use the feature. And as shown later it doesn't bring any performance benefits because the minus almost never arises as a result of rounding and in the few cases when it does it's trivial to suppress by other mechanisms.

We implemented a benchmark in [FORMAT-BENCHMARK] measuring parsing overhead of supporting `'z'` in `formatter::parse` and found that it adds approx. 15% overhead for the common case of parsing a single format specifier (without the `'z'` flag):

```
-------------------------------------------------—
Benchmark         Time           CPU   Iterations
-------------------------------------------------—
parse          7.90 ns       7.82 ns   87966221
parse_z        9.12 ns       9.02 ns   78832380
```

To put these numbers into perspective, formatting a single integer with `format_to` on the same system takes ~20ns which includes format string parsing so the overhead is very significant. This clearly violates "don't pay for what you don't use" philosophy.

§ 3. Locale

P1496 doesn't discuss interaction of the proposed option with locales. However, it mentions that the goal of the proposal is to make the output "unsurprising to most users". This suggests that the use case for this feature is messages displayed to end-users. Such formatting should normally be locale-specific as opposed to locale-independent one that primarily addresses use cases such as text-based serialization, writing output in JSON, XML or other formats ([N4412]), logging, simple non-localized command-line interfaces. In many of the latter cases preserving information and performance are important factors and end-users are not expected to see the output. For this reason we think that if sign suppression was desirable, it should have been a locale-specific option. However we do not explore this idea any further because we don't think there is enough evidence to justify more work on this feature at all.

§ 4. Demotivating example

P1496 proposes adding the `'z'` option to format specifications:

> With the `'z'` option a negative zero after rounding is formatted as a (positive) zero

and provides the following example:

> ```
> format("{0:.0} {0:+.0} {0:-.0} {0: .0}", 0.1) -0 -0 -0 -0
> ```

Unfortunately the example is misleading because the actual output is not `"-0 -0 -0 -0"` as the paper claims but `"-0.1 -0.1 -0.1 -0.1"` so adding the `'z'` option would have no effect. This issue would be easily caught if the proposal was actually implemented or at least the current allegedly problematic behavior tested. The reason why the output is `"-0.1"` is that the default floating-point format is defined as follows in [tab:format.type.float]:

> If `precision` is specified, equivalent to `to_chars(first, last, value, chars_format::general, precision)`

which, in turn, means that the value is formatted in the style of `printf` in the "C" locale with the given precision and the format specifier `'g'`:

> ```
> printf("%.0g", -0.1);
> ```

giving the output of `"-0.1"` because printf produces at least one significant digit in the general format ([C]).

So the only case when the general or default format can produce -0.0 is when the input is -0.0 and it can be trivially suppressed:

> ```
> double x = -0.0;
> auto s = std::format("{}", +x); // s == "0.0"
> ```

Similarly, the `'z'` option would be useless with the `'e'` and `'a'` specifiers and their uppercase counterparts where the negative zero cannot be produced as a result of rounding either, for example:

```
auto s0 = std::format("{:.0e}", -0.1); // s0 == "-1e-01"
auto s1 = std::format("{:.0a}", -0.5); // s1 == "-0x1p-1"
```

In addition to that, a user who finds `-` confusing won't be happy about the output of `'a'` at all, with or without sign.

The fact that nobody cared enough to check the examples that were clearly broken since revision 0 of P1496 published February 2019 is another indication that the motivation to add this feature is very weak.

§ 5. Fixed form

The only case where a negative zero can be produced as a result of rounding is the fixed form enabled by the `'f'` and `'F'` specifiers:

```
auto s = std::format("{:.0f}", -0.1); // s == "-0"
```

However, this case is also trivially addressed, for example:

```
double no_minus(double x) {
  return x >= -0.5 && x < 0 ? 0 : x;
}
auto s = std::format("{:.0f}", no_minus(-0.1)); // s == "0"
```

which, unlike the specifier, doesn't add any overhead when you don't use the feature. It can be easily generalized to arbitrary precision:

```
struct no_minus_zero {
  double value;
  int precision;

  no_minus_zero(double val, int prec) : value(val), precision(prec) {
    auto bound = -0.5 * std::pow(10, -prec);
    if (val >= 0 || val < bound) return;
    if (val > bound || format("{:.{}f}", val, prec).back() == '0')
      value = 0;
  }
};

template <>
struct formatter<no_minus_zero> {
  auto parse(format_parse_context& ctx) { return ctx.begin(); }

  auto format(no_minus_zero nmz, format_context& ctx) {
    return format_to(ctx.out(), "{:.{}f}", nmz.value, nmz.precision);
  }
};
```

Again, this doesn't add any overhead if the feature is not used and most cases require one or two checks when the feature is used. It can be optimized by using a table of bounds and replacing the `format` call with a bound rounding direction check which can also be precomputed but that's not essential.

`no_minus_zero` can be used as follows:

```
auto s = fmt::format("{}", no_minus_zero(-0.001, 2)) // s == "0.00"
```

This will likely be faster than `fmt::format("{:z.2f}")` because there is less parsing of format specifiers and fewer runtime checks.

So we can clearly see that the analysis in The Problem section of P1496 is incorrect: negative zero can never appear as a result of rounding with the default, `'g'`, `'G'`, `'e'`, `'E'`, `'a'`, and `'A'` specifiers and can be trivially handled in the case of `'f'` and `'F'` specifiers due to the nature of the fixed format (the range that rounds to -0 is easy to compute).

§ 6. Conclusion

Considering that there is almost zero evidence that sign suppression is a problem worth solving, motivating examples in P1496 are incorrect and this new feature will add significant overhead to calls to `std::format` and uses of built-in `formatter` specializations even if not used among other problems and the fact that it can be easily implemented by other means, we think that it shouldn't be standardized. If there is still a strong desire to have this feature, it can be added via a separate formatter specialization not penalizing other cases.

§ 7. Acknowledgements

We would like to thank Peter Brett for bringing to our attention the fact that negative zero can never appear as a result of rounding in most formats.

§ References

§ Informative References

[C]
ISO/IEC 9899:2011 Information technology — Programming languages — C.

[D3-FORMAT]
d3-format: Format numbers for human consumption. URL: https://github.com/d3/d3-format

[FMT]
The fmt library. URL: https://github.com/fmtlib/fmt

[FOLLY-FORMAT]
Folly Format. URL: https://github.com/facebook/folly/blob/master/folly/docs/Format.md

[FORMAT-BENCHMARK]
A collection of formatting benchmarks. URL: https://github.com/fmtlib/format-benchmark

[N4412]
Jens Maurer. Shortcomings of iostreams. URL: http://open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4412.html

[P1496]
Alan Talbot; Jorg Brown. Formatting of Negative Zero. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1496r1.pdf

[US227]

Add "z" format modifier to avoid sign for zero-display numbers. URL: https://github.com/cplusplus/nbballot/issues/224